

LAPORAN TUGAS 3 SISTEM OPERASI

Threads & Concurrency



Dosen Pembimbing:

Triawan Adi Cahyanto M.Kom

Dibuat Oleh:

Nadianur Anisa (2410651026)

TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

TUGAS 1 KALKULATOR PARALEL

Buat program yang:

1. Menerima input 2 angka dari user
2. Menghitung penjumlahan, pengurangan, perkalian, dan pembagian
3. Setiap operasi dikerjakan oleh thread berbeda
4. Tampilkan semua hasil setelah semua thread selesai

TUJUAN:

Tujuan dari percobaan ini adalah untuk membuat program kalkulator sederhana yang dapat menjalankan empat operasi aritmatika (penjumlahan, pengurangan, perkalian, dan pembagian) secara bersamaan menggunakan konsep multithreading. Dengan cara ini, setiap operasi bisa dikerjakan oleh thread yang berbeda secara paralel.

DASAR TEORI:

Thread adalah unit kecil dari proses yang bisa dijalankan secara bersamaan. Dengan menggunakan beberapa thread, sebuah program dapat mengerjakan beberapa tugas di waktu yang hampir bersamaan, sehingga lebih efisien dibanding dijalankan secara berurutan.

Dalam bahasa C, pembuatan thread bisa dilakukan menggunakan pustaka pthread. Fungsi utama yang digunakan yaitu:

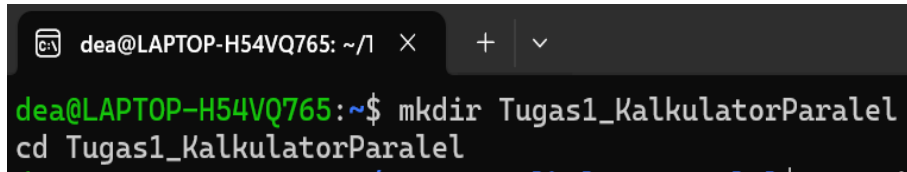
- `pthread_create()` untuk membuat thread baru.
- `pthread_join()` untuk menunggu thread selesai dieksekusi.

Multithreading sering digunakan untuk memecah tugas besar menjadi beberapa bagian kecil agar dapat dikerjakan lebih cepat, terutama pada sistem yang memiliki lebih dari satu inti prosesor (multi-core).

PERCOBAAN TUGAS 1 KALKULATOR PARALEL

1. Buat folder

Buka terminal, lalu buat folder baru:

A terminal window with a dark background. The prompt is 'dea@LAPTOP-H54VQ765: ~/1'. The user has entered 'mkdir Tugas1_KalkulatorParalel' and 'cd Tugas1_KalkulatorParalel'.

```
dea@LAPTOP-H54VQ765: ~/1 × + v
dea@LAPTOP-H54VQ765:~$ mkdir Tugas1_KalkulatorParalel
cd Tugas1_KalkulatorParalel
```

2. Buat file program C:

A terminal window with a dark background. The prompt is 'dea@LAPTOP-H54VQ765: ~/Tugas1_KalkulatorParalel'. The user has entered 'nano kalkulator_parallel.c'.

```
dea@LAPTOP-H54VQ765:~/Tugas1_KalkulatorParalel$ nano kalkulator_
parallel.c
```

3. Kode Program

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
// Struktur untuk menyimpan dua angka
```

```
typedef struct {
```

```
    double a;
```

```
    double b;
```

```
} Angka;
```

```
// Variabel global untuk hasil
```

```
double hasil_tambah, hasil_kurang, hasil_kali, hasil_bagi;
```

```
// Fungsi thread untuk penjumlahan
```

```
void* tambah(void* arg) {
```

```
    Angka* data = (Angka*)arg;
```

```
    hasil_tambah = data->a + data->b;
```

```
    pthread_exit(NULL);
```

```
}
```

```
// Fungsi thread untuk pengurangan
```

```
void* kurang(void* arg) {
```

```
    Angka* data = (Angka*)arg;
```

```
    hasil_kurang = data->a - data->b;
```

```

    pthread_exit(NULL);
}

// Fungsi thread untuk perkalian
void* kali(void* arg) {
    Angka* data = (Angka*)arg;
    hasil_kali = data->a * data->b;
    pthread_exit(NULL);
}

// Fungsi thread untuk pembagian
void* bagi(void* arg) {
    Angka* data = (Angka*)arg;
    if (data->b != 0)
        hasil_bagi = data->a / data->b;
    else
        hasil_bagi = 0; // hindari pembagian dengan nol
    pthread_exit(NULL);
}

int main() {
    pthread_t t1, t2, t3, t4;
    Angka data;

    // Input dari user
    printf("=== Kalkulator Paralel ===\n");
    printf("Masukkan angka pertama: ");
    scanf("%lf", &data.a);
    printf("Masukkan angka kedua : ");
    scanf("%lf", &data.b);

    // Buat thread untuk tiap operasi
    pthread_create(&t1, NULL, tambah, &data);
    pthread_create(&t2, NULL, kurang, &data);
    pthread_create(&t3, NULL, kali, &data);
    pthread_create(&t4, NULL, bagi, &data);

```

```

// Tunggu semua thread selesai
pthread_join(t1, NULL);
pthread_join(t2, NULL);
pthread_join(t3, NULL);
pthread_join(t4, NULL);

// Tampilkan hasil
printf("\n=== Hasil Perhitungan ===\n");
printf("Penjumlahan : %.2lf\n", hasil_tambah);
printf("Pengurangan : %.2lf\n", hasil_kurang);
printf("Perkalian   : %.2lf\n", hasil_kali);
printf("Pembagian    : %.2lf\n", hasil_bagi);

return 0;
}

```

4. Menyimpan file

Tekan Ctrl + O lalu Enter untuk menyimpan

Tekan Ctrl + X untuk keluar dari nano

5. Compile dan jalankan program

```

dea@LAPTOP-H54VQ765:~/Tugas1_KalkulatorParalel$ gcc kalkulator_p
aralel.c -o kalkulator_paralel -lpthread
dea@LAPTOP-H54VQ765:~/Tugas1_KalkulatorParalel$ ./kalkulator_par
alel

```

6. Tampilan hasil output

```

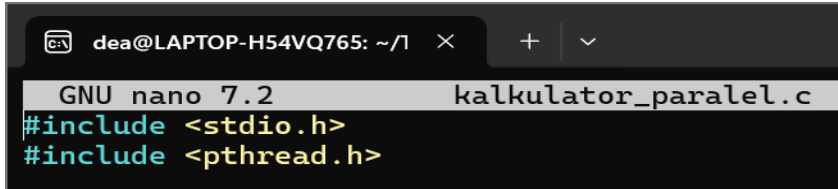
=== Kalkulator Paralel ===
Masukkan angka pertama: 10
Masukkan angka kedua  : 5

=== Hasil Perhitungan ===
Penjumlahan : 15.00
Pengurangan : 5.00
Perkalian   : 50.00
Pembagian    : 2.00
dea@LAPTOP-H54VQ765:~/Tugas1_KalkulatorParalel$ |

```

IMPLEMENTASI TUGAS 1 KALKUKATOR PARALEL:

A. Bagian #include



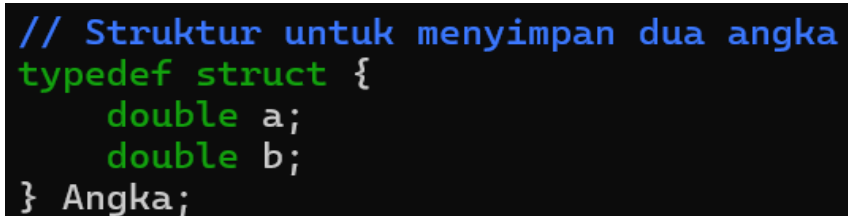
```
dea@LAPTOP-H54VQ765: ~/1 × + v
GNU nano 7.2 kalkulator_paralel.c
#include <stdio.h>
#include <pthread.h>
```

Penjelasan:

Pada bagian ini, kita mengimpor dua pustaka. *stdio.h* digunakan supaya program bisa memakai fungsi input-output seperti *printf* dan *scanf*.

Sedangkan *pthread.h* adalah pustaka yang berisi fungsi-fungsi untuk membuat dan mengatur *thread* di Bahasa C.

B. Struktur Data



```
// Struktur untuk menyimpan dua angka
typedef struct {
    double a;
    double b;
} Angka;
```

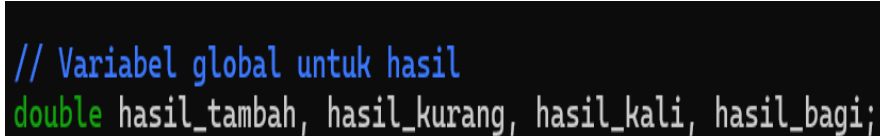
Penjelasan:

Struktur ini berfungsi untuk menyimpan dua buah angka yang diinput oleh pengguna.

Keduanya bertipe *double* supaya bisa digunakan untuk perhitungan bilangan desimal juga.

Struktur ini nantinya dikirim ke setiap *thread* agar semua operasi (tambah, kurang, kali, bagi) menggunakan data yang sama.

C. Variabel Global



```
// Variabel global untuk hasil
double hasil_tambah, hasil_kurang, hasil_kali, hasil_bagi;
```

Penjelasan:

Variabel ini disiapkan untuk menampung hasil dari masing-masing operasi aritmatika.

Dibuat secara global agar semua fungsi *thread* bisa mengaksesnya dan menyimpan hasil perhitungannya masing-masing.

D. Fungsi Penjumlahan

```
// Fungsi thread untuk penjumlahan
void* tambah(void* arg) {
    Angka* data = (Angka*)arg;
    hasil_tambah = data->a + data->b;
    pthread_exit(NULL);
}
```

Penjelasan:

Fungsi ini akan dijalankan oleh *thread* pertama.

Parameter *arg* berisi data yang dikirim dari **main()*, kemudian dikonversi ke tipe *Angka*. Setelah itu dilakukan operasi $a + b$, dan hasilnya disimpan ke variabel *hasil_tambah* bagian `pthread_exit(NULL)` menandakan bahwa thread selesai menjalankan tugasnya.

E. Fungsi Pengurangan

```
// Fungsi thread untuk pengurangan
void* kurang(void* arg) {
    Angka* data = (Angka*)arg;
    hasil_kurang = data->a - data->b;
    pthread_exit(NULL);
}
```

Penjelasan:

Fungsi ini mirip dengan fungsi sebelumnya, hanya saja operasi yang dilakukan adalah $a - b$.

Thread ini bertanggung jawab khusus untuk menghitung hasil pengurangan.

F. Fungsi Perkalian

```
// Fungsi thread untuk perkalian
void* kali(void* arg) {
    Angka* data = (Angka*)arg;
    hasil_kali = data->a * data->b;
    pthread_exit(NULL);
}
```

Penjelasan:

Bagian ini digunakan untuk menghitung hasil perkalian antara dua angka.

Thread ketiga menjalankan fungsi ini agar proses perkalian dilakukan secara paralel dengan operasi lainnya.

G. Fungsi Pembagian

```
// Fungsi thread untuk pembagian
void* bagi(void* arg) {
    Angka* data = (Angka*)arg;
    if (data->b != 0)
        hasil_bagi = data->a / data->b;
    else
        hasil_bagi = 0; // hindari pembagian dengan nol
    pthread_exit(NULL);
}
```

Penjelasan:

Fungsi ini menghitung hasil pembagian.

Sebelum membagi, ada pengecekan if ($\text{data} \rightarrow \text{b} \neq 0$) untuk memastikan angka pembagi tidak nol.

Kalau nol, hasilnya diberi nilai 0 supaya program tidak error.

Thread keempat akan menjalankan fungsi ini.

H. Fungsi main()

a. Deklarasi Thread dan Variabel

```
int main() {
    pthread_t t1, t2, t3, t4;
    Angka data;
```

Di sini dibuat empat variabel *thread* (t1, t2, t3 dan t4) untuk masing-masing operasi, serta satu variable data untuk menyimpan input angka dari pengguna.

b. Input dari Pengguna

```
// Input dari user
printf("=== Kalkulator Paralel ===\n");
printf("Masukkan angka pertama: ");
scanf("%lf", &data.a);
printf("Masukkan angka kedua : ");
scanf("%lf", &data.b);
```

Program meminta dua angka dari pengguna melalui terminal dan menyimpannya di dalam data.

c. Membuat Thread

```
// Buat thread untuk tiap operasi
pthread_create(&t1, NULL, tambah, &data);
pthread_create(&t2, NULL, kurang, &data);
pthread_create(&t3, NULL, kali, &data);
pthread_create(&t4, NULL, bagi, &data);
```

- Empat baris ini membuat empat thread berbeda.
- Setiap thread menjalankan satu fungsi operasi matematika (tambah, kurang, kali, bagi) dengan membawa data yang sama (&data).
- Karena thread dijalankan bersamaan, keempat operasi ini bisa berlangsung paralel.

d. Menunggu Semua Thread Selesai

```
// Tunggu semua thread selesai
pthread_join(t1, NULL);
pthread_join(t2, NULL);
pthread_join(t3, NULL);
pthread_join(t4, NULL);
```

- Bagian ini digunakan agar program utama menunggu semua thread selesai terlebih dahulu.
- Kalau tidak ada pthread_join, bisa saja hasil belum selesai dihitung tapi program sudah berlanjut menampilkan output.

e. Menampilkan Hasil

```
// Tampilkan hasil
printf("\n=== Hasil Perhitungan ===\n");
printf("Penjumlahan : %.2lf\n", hasil_tambah);
printf("Pengurangan : %.2lf\n", hasil_kurang);
printf("Perkalian : %.2lf\n", hasil_kali);
printf("Pembagian : %.2lf\n", hasil_bagi);

return 0;
}
```

Setelah semua thread selesai, hasil perhitungan ditampilkan ke layar.

Masing-masing hasil sudah diisi oleh thread yang sesuai.

TUGAS 2 FILE PROCESSING

Buat program yang:

1. Membaca file teks
2. Menghitung jumlah kata menggunakan 2 thread (bagi file jadi 2 bagian)
3. Gabungkan hasilnya
4. Bandingkan waktu eksekusi dengan versi single-thread.

TUJUAN:

1. Menerapkan konsep File Processing untuk membaca isi file teks.
2. Memahami penggunaan thread (pthread) dalam mempercepat proses komputasi.
3. Membandingkan kinerja single-thread dan multi-thread dalam menghitung jumlah kata.

DASAR TEORI:

1. File Processing (Pemrosesan File)
Merupakan proses membaca atau menulis data ke dalam file menggunakan program. Dalam C, file diakses dengan fungsi fopen(), fread(), fwrite(), dan fclose().
2. Multithreading
Adalah teknik untuk menjalankan beberapa proses kecil (thread) secara bersamaan dalam satu program. Dengan membagi pekerjaan menjadi beberapa thread, program bisa berjalan lebih cepat terutama pada data besar.
3. pthread (POSIX Thread)
Merupakan library di C untuk membuat dan mengelola thread. Fungsi utamanya antara lain:
 - pthread_create() → membuat thread baru
 - pthread_join() → menunggu thread selesai
 - pthread_exit() → keluar dari thread
4. Perbandingan Single vs Multi-thread
 - *Single-thread*: semua proses dijalankan secara berurutan.
 - *Multi-thread*: beberapa proses dijalankan bersamaan, bisa mempercepat waktu eksekusi.

PERCOBAAN TUGAS 2 FILE PROCESSING

1. Buat file program C



```
dea@LAPTOP-H54VQ765:~/Tugas1_KalkulatorParalel$ nano file_processing.c
```

2. Kode program

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <string.h>
#include <time.h>

#define MAX_TEXT 10000

// Struktur data untuk menyimpan bagian teks yang akan dihitung
typedef struct {
    char *text;
    int start;
    int end;
    int count;
} ThreadData;

// Fungsi untuk menghitung jumlah kata di potongan teks
void *count_words(void *arg) {
    ThreadData *data = (ThreadData *)arg;
    int count = 0;
    int inWord = 0;

    for (int i = data->start; i < data->end; i++) {
        char c = data->text[i];
        if (c == ' ' || c == '\n' || c == '\t') {
            inWord = 0;
        } else if (!inWord) {
            inWord = 1;
            count++;
        }
    }
    data->count = count;
    pthread_exit(NULL);
}
```

```

// Fungsi menghitung kata secara single-thread
int count_words_single(char *text) {
    int count = 0;
    int inWord = 0;
    for (int i = 0; text[i] != '\0'; i++) {
        if (text[i] == ' ' || text[i] == '\n' || text[i] == '\t')
            inWord = 0;
        else if (!inWord) {
            inWord = 1;
            count++;
        }
    }
    return count;
}

int main() {
    FILE *fp = fopen("input.txt", "r");
    if (!fp) {
        printf("File tidak ditemukan!\n");
        return 1;
    }

    char *text = malloc(MAX_TEXT);
    fread(text, 1, MAX_TEXT, fp);
    fclose(fp);

    int len = strlen(text);
    int mid = len / 2;

    pthread_t t1, t2;
    ThreadData data1 = {text, 0, mid, 0};
    ThreadData data2 = {text, mid, len, 0};

    // Waktu single-thread
    clock_t start_single = clock();
    int total_single = count_words_single(text);
    clock_t end_single = clock();

    // Waktu multi-thread

```

```

clock_t start_multi = clock();
pthread_create(&t1, NULL, count_words, &data1);
pthread_create(&t2, NULL, count_words, &data2);
pthread_join(t1, NULL);
pthread_join(t2, NULL);
int total_multi = data1.count + data2.count;
clock_t end_multi = clock();

double time_single = (double)(end_single - start_single) / CLOCKS_PER_SEC;
double time_multi = (double)(end_multi - start_multi) / CLOCKS_PER_SEC;

printf("=== HASIL ===\n");
printf("Jumlah kata (Single Thread): %d\n", total_single);
printf("Jumlah kata (Multi Thread): %d\n", total_multi);
printf("Waktu Single Thread: %.6f detik\n", time_single);
printf("Waktu Multi Thread: %.6f detik\n", time_multi);

free(text);
return 0;
}

```

3. Buat file teks input

```

dea@LAPTOP-H54VQ765:~/Tugas1_KalkulatorParalel$ nano input.txt

```

→ Isi nano input.txt

Ini adalah contoh file teks yang digunakan untuk menghitung jumlah kata menggunakan thread di bahasa C.

Multithreading bisa mempercepat proses jika file berukuran besar.

4. Kompilasi dan jalankan program

```

dea@LAPTOP-H54VQ765:~/Tugas1_KalkulatorParalel$ gcc file_processing.c -o file_processing -lpthread
./file_processing

```

5. Tampilan hasil output

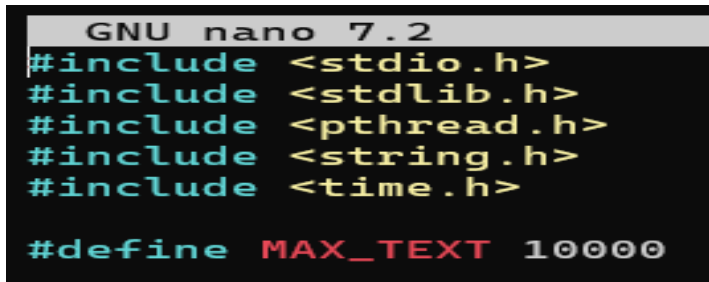
```

dea@LAPTOP-H54VQ765:~/Tugas1_KalkulatorParalel$ nano file_processing.c
dea@LAPTOP-H54VQ765:~/Tugas1_KalkulatorParalel$ nano input.txt
dea@LAPTOP-H54VQ765:~/Tugas1_KalkulatorParalel$ gcc file_processing.c -o file_processing -lpthread
./file_processing
=== HASIL ===
Jumlah kata (Single Thread): 24
Jumlah kata (Multi Thread): 25
Waktu Single Thread: 0.000007 detik
Waktu Multi Thread: 0.005684 detik

```

IMPLEMENTASI TUGAS 2 FILE PROCESSING

a. Library dan Konstanta



```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <string.h>
#include <time.h>

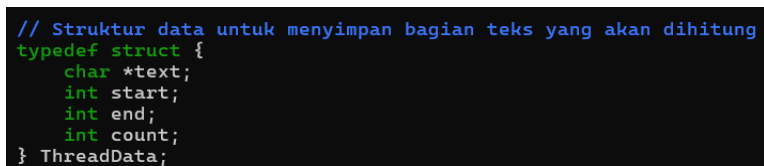
#define MAX_TEXT 10000
```

Penjelasan:

Import library yang dibutuhkan:

- `stdio.h` → input/output
- `stdlib.h` → fungsi umum (`malloc`, `exit`)
- `pthread.h` → library untuk thread
- `string.h` → manipulasi string
- `time.h` → menghitung waktu eksekusi
- `MAX_TEXT` batas maksimum ukuran teks yang dibaca (10.000 karakter)

b. Struktur Data



```
// Struktur data untuk menyimpan bagian teks yang akan dihitung
typedef struct {
    char *text;
    int start;
    int end;
    int count;
} ThreadData;
```

Penjelasan:

Struktur ini menyimpan data yang akan diproses oleh setiap thread:

- `text` → isi file teks
- `start` dan `end` → batas indeks teks yang akan dihitung
- `count` → hasil jumlah kata dari potongan teks

c. Fungsi Thread (count_words)

```
// Fungsi untuk menghitung jumlah kata di potongan teks
void *count_words(void *arg) {
    ThreadData *data = (ThreadData *)arg;
    int count = 0;
    int inWord = 0;

    for (int i = data->start; i < data->end; i++) {
        char c = data->text[i];
        if (c == ' ' || c == '\n' || c == '\t') {
            inWord = 0;
        } else if (!inWord) {
            inWord = 1;
            count++;
        }
    }
    data->count = count;
    pthread_exit(NULL);
}
```

Penjelasan:

Fungsi ini dijalankan oleh setiap thread:

- Loop dari posisi start sampai end untuk menghitung kata.
- Variabel inWord digunakan agar satu kata tidak dihitung dua kali.
- Hasil jumlah kata disimpan di data->count.
- pthread_exit() menandakan thread selesai bekerja.

d. Fungsi Single-thread

```
// Fungsi menghitung kata secara single-thread
int count_words_single(char *text) {
    int count = 0;
    int inWord = 0;
    for (int i = 0; text[i] != '\0'; i++) {
        if (text[i] == ' ' || text[i] == '\n' || text[i] == '\t') {
            inWord = 0;
        } else if (!inWord) {
            inWord = 1;
            count++;
        }
    }
    return count;
}
```

Penjelasan:

Hanya menghitung jumlah kata dari awal sampai akhir teks secara langsung.

e. Fungsi Utama (main)

```
int main() {  
    FILE *fp = fopen("input.txt", "r");  
    if (!fp) {  
        printf("File tidak ditemukan!\n");  
        return 1;  
    }  
  
    char *text = malloc(MAX_TEXT);  
    fread(text, 1, MAX_TEXT, fp);  
    fclose(fp);  
}
```

Penjelasan:

Membuka file input.txt, membaca seluruh isinya, lalu menutup file.

Teks disimpan di variabel text.

```
int len = strlen(text);  
int mid = len / 2;
```

Penjelasan: int len = strlen(text);

int mid = len / 2;

Menentukan panjang teks dan titik tengah agar bisa dibagi untuk dua thread.

```
pthread_t t1, t2;  
ThreadData data1 = {text, 0, mid, 0};  
ThreadData data2 = {text, mid, len, 0};
```

Penjelasan: pthread_t t1, t2;

ThreadData data1 = {text, 0, mid, 0};

ThreadData data2 = {text, mid, len, 0};

Membuat dua thread (t1 dan t2) dan dua data bagian teks (bagian pertama dan kedua).

```
// Waktu single-thread
clock_t start_single = clock();
int total_single = count_words_single(text);
clock_t end_single = clock();
```

Penjelasan:

Mengukur waktu eksekusi versi single-thread.

```
// Waktu multi-thread
clock_t start_multi = clock();
pthread_create(&t1, NULL, count_words, &data1);
pthread_create(&t2, NULL, count_words, &data2);
pthread_join(t1, NULL);
pthread_join(t2, NULL);
int total_multi = data1.count + data2.count;
clock_t end_multi = clock();
```

Penjelasan:

Membuat dua thread untuk menghitung bagian teks masing-masing.

Setelah selesai (pthread_join), hasil dari kedua thread dijumlahkan.

```
double time_single = (double)(end_single - start_single) / CLOCKS_PER_SEC;
double time_multi = (double)(end_multi - start_multi) / CLOCKS_PER_SEC;
```

Penjelasan:

Mengubah selisih waktu ke dalam satuan detik.

```

printf("=== HASIL ===\n");
printf("Jumlah kata (Single Thread): %d\n", total_single);
printf("Jumlah kata (Multi Thread): %d\n", total_multi);
printf("Waktu Single Thread: %.6f detik\n", time_single);
printf("Waktu Multi Thread: %.6f detik\n", time_multi);

free(text);
return 0;
}

```

Penjelasan:

Menampilkan hasil perbandingan jumlah kata dan waktu eksekusi.

f. Hasil output

```

=== HASIL ===
Jumlah kata (Single Thread): 24
Jumlah kata (Multi Thread): 25
Waktu Single Thread: 0.000007 detik
Waktu Multi Thread: 0.005684 detik

```

Penjelasan:

- Jumlah kata (Single Thread): 24
Artinya ketika program menghitung seluruh isi file menggunakan satu proses (tanpa dibagi dua), ditemukan 24 kata.
- Jumlah kata (Multi Thread): 25
Hasil sedikit berbeda (lebih 1 kata) karena saat file dibagi dua, kemungkinan ada kata yang terpotong di bagian tengah (tepat di batas pembagian), sehingga dihitung dua kali oleh thread pertama dan kedua.
- Waktu Single Thread: 0.000007 detik
Waktu sangat cepat karena file kecil, jadi perhitungan satu thread berjalan efisien.
- Waktu Multi Thread: 0.005684 detik
Justru lebih lambat karena data kecil dan proses pembuatan thread membutuhkan waktu tambahan (overhead). Multithreading baru terasa lebih cepat jika file yang diproses sangat besar.

TUGAS 3 PRODUCER-CONSUMER

Buat simulasi producer-consumer dengan:

1. Thread producer yang menghasilkan angka random
2. Thread consumer yang memproses angka tersebut
3. Gunakan mutex untuk sinkronisasi
4. Buffer maksimal 10 item

TUJUAN:

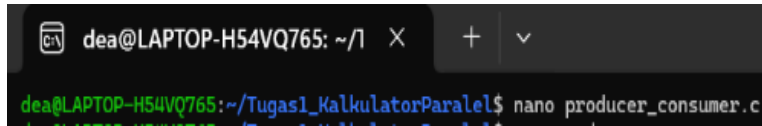
- Memahami konsep *Producer-Consumer* menggunakan *thread*.
- Mempelajari sinkronisasi thread dengan *mutex* dan *condition variable*.
- Mengimplementasikan buffer terbatas untuk pertukaran data antar thread.

DASAR TEORI:

- **Producer-Consumer Problem:**
Masalah klasik pada concurrent programming di mana producer menghasilkan data dan meletakkannya ke buffer, sementara *consumer* mengambil data dari buffer. Buffer memiliki kapasitas terbatas, sehingga perlu sinkronisasi agar producer tidak menimpa buffer penuh dan consumer tidak mengambil dari buffer kosong.
- **Mutex:**
Digunakan untuk mencegah dua thread mengakses data bersama pada saat bersamaan sehingga tidak terjadi race condition.
- **Condition Variable:**
Digunakan untuk memberi sinyal thread lain agar menunggu atau melanjutkan eksekusi ketika kondisi tertentu (buffer penuh/kosong) terpenuhi.
- **Buffer:**
Tempat penyimpanan sementara data antara producer dan consumer. Pada praktikum ini buffer maksimal 10 item.

PERCOBAAN TUGAS 3 PRODUCER-CONSUMER

1. Buat file program C



```
dea@LAPTOP-H54VQ765: ~/1 X + v
dea@LAPTOP-H54VQ765:~/Tugas1_KalkulatorParalel$ nano producer_consumer.c
```

2. Kode program

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define BUFFER_SIZE 10 // buffer maksimal 10 item
#define PRODUCE_COUNT 20 // total item yang akan diproduksi

int buffer[BUFFER_SIZE];
int count = 0; // jumlah item dalam buffer

pthread_mutex_t mutex;
pthread_cond_t not_full;
pthread_cond_t not_empty;

// Fungsi producer: menghasilkan angka random dan memasukkan ke buffer
void* producer(void* arg) {
    for (int i = 0; i < PRODUCE_COUNT; i++) {
        int item = rand() % 100; // angka random 0-99

        pthread_mutex_lock(&mutex);

        // jika buffer penuh, tunggu hingga ada ruang
        while (count == BUFFER_SIZE) {
            pthread_cond_wait(&not_full, &mutex);
        }

        // masukkan item ke buffer
        buffer[count++] = item;
        printf("Producer menghasilkan: %d (jumlah item: %d)\n", item, count);

        pthread_cond_signal(&not_empty); // beri sinyal ke consumer
        pthread_mutex_unlock(&mutex);
    }
}
```

```

        sleep(1); // simulasi waktu produksi
    }
    pthread_exit(NULL);
}

// Fungsi consumer: mengambil dan memproses angka dari buffer
void* consumer(void* arg) {
    for (int i = 0; i < PRODUCE_COUNT; i++) {
        pthread_mutex_lock(&mutex);

        // jika buffer kosong, tunggu hingga ada item
        while (count == 0) {
            pthread_cond_wait(&not_empty, &mutex);
        }

        // ambil item dari buffer
        int item = buffer[--count];
        printf("Consumer memproses : %d (sisanya item: %d)\n", item, count);

        pthread_cond_signal(&not_full); // beri sinyal ke producer
        pthread_mutex_unlock(&mutex);

        sleep(2); // simulasi waktu konsumsi
    }
    pthread_exit(NULL);
}

int main() {
    pthread_t prod_thread, cons_thread;

    // inisialisasi mutex & condition variable
    pthread_mutex_init(&mutex, NULL);
    pthread_cond_init(&not_full, NULL);
    pthread_cond_init(&not_empty, NULL);

    // buat thread producer dan consumer
    pthread_create(&prod_thread, NULL, producer, NULL);
    pthread_create(&cons_thread, NULL, consumer, NULL);

```

```

// tunggu kedua thread selesai
pthread_join(prod_thread, NULL);
pthread_join(cons_thread, NULL);

// hapus resource
pthread_mutex_destroy(&mutex);
pthread_cond_destroy(&not_full);
pthread_cond_destroy(&not_empty);

printf("\nProduksi dan konsumsi selesai.\n");
return 0;
}

```

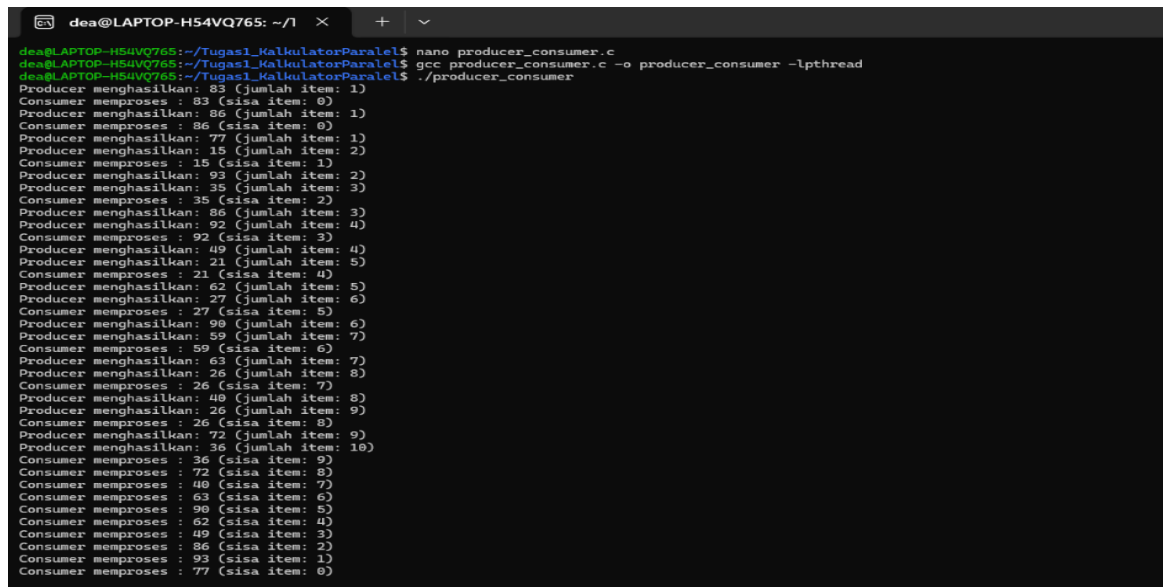
3. Simpan dengan tekan:
CTRL + O ENTER, lalu CTRL + X untuk keluar
4. Kompilasi dan jalankan program

```

dea@LAPTOP-H54VQ765:~/Tugas1_KalkulatorParalel$ gcc producer_consumer.c -o producer_consumer -lpthread
dea@LAPTOP-H54VQ765:~/Tugas1_KalkulatorParalel$ ./producer_consumer

```

5. Tampilan hasil output



```

dea@LAPTOP-H54VQ765: ~/1 X + v
dea@LAPTOP-H54VQ765:~/Tugas1_KalkulatorParalel$ nano producer_consumer.c
dea@LAPTOP-H54VQ765:~/Tugas1_KalkulatorParalel$ gcc producer_consumer.c -o producer_consumer -lpthread
dea@LAPTOP-H54VQ765:~/Tugas1_KalkulatorParalel$ ./producer_consumer
Producer menghasilkan: 83 (jumlah item: 1)
Consumer memproses : 83 (sisa item: 0)
Producer menghasilkan: 86 (jumlah item: 1)
Consumer memproses : 86 (sisa item: 0)
Producer menghasilkan: 77 (jumlah item: 1)
Producer menghasilkan: 15 (jumlah item: 2)
Consumer memproses : 15 (sisa item: 1)
Producer menghasilkan: 93 (jumlah item: 2)
Producer menghasilkan: 35 (jumlah item: 3)
Consumer memproses : 35 (sisa item: 2)
Producer menghasilkan: 86 (jumlah item: 3)
Producer menghasilkan: 92 (jumlah item: 4)
Consumer memproses : 92 (sisa item: 3)
Producer menghasilkan: 49 (jumlah item: 4)
Producer menghasilkan: 21 (jumlah item: 5)
Consumer memproses : 21 (sisa item: 4)
Producer menghasilkan: 62 (jumlah item: 5)
Producer menghasilkan: 27 (jumlah item: 6)
Consumer memproses : 27 (sisa item: 5)
Producer menghasilkan: 90 (jumlah item: 6)
Producer menghasilkan: 59 (jumlah item: 7)
Consumer memproses : 59 (sisa item: 6)
Producer menghasilkan: 63 (jumlah item: 7)
Producer menghasilkan: 26 (jumlah item: 8)
Consumer memproses : 26 (sisa item: 7)
Producer menghasilkan: 40 (jumlah item: 8)
Producer menghasilkan: 26 (jumlah item: 9)
Consumer memproses : 26 (sisa item: 8)
Producer menghasilkan: 72 (jumlah item: 9)
Producer menghasilkan: 36 (jumlah item: 10)
Consumer memproses : 36 (sisa item: 9)
Consumer memproses : 72 (sisa item: 8)
Consumer memproses : 40 (sisa item: 7)
Consumer memproses : 63 (sisa item: 6)
Consumer memproses : 90 (sisa item: 5)
Consumer memproses : 62 (sisa item: 4)
Consumer memproses : 49 (sisa item: 3)
Consumer memproses : 86 (sisa item: 2)
Consumer memproses : 93 (sisa item: 1)
Consumer memproses : 77 (sisa item: 0)

```

IMPLEMENTASI TUGAS 3 PRODUCER-CONSUMER

a. Header dan Definisi

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define BUFFER_SIZE 10 // buffer maksimal 10 item
#define PRODUCE_COUNT 20 // total item yang akan diproduksi
```

Penjelasan:

- Mengimpor library standar, pthread untuk thread, unistd untuk fungsi sleep.
- BUFFER_SIZE → kapasitas buffer maksimal 10 item.
- PRODUCE_COUNT → total item yang akan diproduksi dan dikonsumsi.

b. Variabel Global

```
int buffer[BUFFER_SIZE];
int count = 0; // jumlah item dalam buffer

pthread_mutex_t mutex;
pthread_cond_t not_full;
pthread_cond_t not_empty;
```

Penjelasan:

- buffer[] → array untuk menyimpan data.
- count → jumlah item dalam buffer saat ini.
- mutex → untuk sinkronisasi akses buffer.
- not_full & not_empty → memberi sinyal saat buffer penuh atau kosong.

c. Fungsi Producer

```
// Fungsi producer: menghasilkan angka random dan memasukkan ke buffer
void* producer(void* arg) {
    for (int i = 0; i < PRODUCE_COUNT; i++) {
        int item = rand() % 100; // angka random 0-99

        pthread_mutex_lock(&mutex);

        // jika buffer penuh, tunggu hingga ada ruang
        while (count == BUFFER_SIZE) {
            pthread_cond_wait(&not_full, &mutex);
        }

        // masukkan item ke buffer
        buffer[count++] = item;
        printf("Producer menghasilkan: %d (jumlah item: %d)\n", item, count);

        pthread_cond_signal(&not_empty); // beri sinyal ke consumer
        pthread_mutex_unlock(&mutex);

        sleep(1); // simulasi waktu produksi
    }
    pthread_exit(NULL);
}
```

Penjelasan:

- Menghasilkan angka random 0–99.
- Mengunci buffer menggunakan *mutex*.
- Jika buffer penuh → tunggu sampai ada ruang (*pthread_cond_wait*).
- Menambahkan item ke buffer dan memberi sinyal ke consumer (*pthread_cond_signal*).

- Melepas *mutex* dan *sleep* untuk simulasi waktu produksi.

d. Fungsi Consumer

```
// Fungsi consumer: mengambil dan memproses angka dari buffer
void* consumer(void* arg) {
    for (int i = 0; i < PRODUCE_COUNT; i++) {
        pthread_mutex_lock(&mutex);

        // jika buffer kosong, tunggu hingga ada item
        while (count == 0) {
            pthread_cond_wait(&not_empty, &mutex);
        }

        // ambil item dari buffer
        int item = buffer[--count];
        printf("Consumer memproses : %d (sisa item: %d)\n", item, count);

        pthread_cond_signal(&not_full); // beri sinyal ke producer
        pthread_mutex_unlock(&mutex);

        sleep(2); // simulasi waktu konsumsi
    }
    pthread_exit(NULL);
}
```

Penjelasan:

- Mengunci buffer agar aman diakses.
- Jika buffer kosong → tunggu sampai ada item (`pthread_cond_wait`).
- Mengambil item dari buffer, menampilkan hasil, memberi sinyal ke producer.
- Melepas mutex dan sleep untuk simulasi waktu konsumsi.

e. Fungsi Main

```
int main() {
    pthread_t prod_thread, cons_thread;

    // inisialisasi mutex & condition variable
    pthread_mutex_init(&mutex, NULL);
    pthread_cond_init(&not_full, NULL);
    pthread_cond_init(&not_empty, NULL);

    // buat thread producer dan consumer
    pthread_create(&prod_thread, NULL, producer, NULL);
    pthread_create(&cons_thread, NULL, consumer, NULL);

    // tunggu kedua thread selesai
    pthread_join(prod_thread, NULL);
    pthread_join(cons_thread, NULL);

    // hapus resource
    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&not_full);
    pthread_cond_destroy(&not_empty);

    printf("\nProduksi dan konsumsi selesai.\n");
    return 0;
}
```

Penjelasan:

- Membuat thread producer dan consumer.
- Menunggu kedua thread selesai (`pthread_join`).
- Membersihkan *mutex* dan *condition variable*.
- Menampilkan pesan akhir setelah semua item diproduksi dan dikonsumsi.

f. Penjelasan Hasil Output

```
dea@LAPTOP-H54VQ765: ~/Tugas1_KalkulatorParalel$ nano producer_consumer.c
dea@LAPTOP-H54VQ765:~/Tugas1_KalkulatorParalel$ gcc producer_consumer.c -o producer_consumer -lpthread
dea@LAPTOP-H54VQ765:~/Tugas1_KalkulatorParalel$ ./producer_consumer
Producer menghasilkan: 83 (jumlah item: 1)
Consumer memproses : 83 (sisitem: 0)
Producer menghasilkan: 86 (jumlah item: 1)
Consumer memproses : 86 (sisitem: 0)
Producer menghasilkan: 77 (jumlah item: 1)
Producer menghasilkan: 15 (jumlah item: 2)
Consumer memproses : 15 (sisitem: 1)
Producer menghasilkan: 93 (jumlah item: 2)
Producer menghasilkan: 35 (jumlah item: 3)
Consumer memproses : 35 (sisitem: 2)
Producer menghasilkan: 86 (jumlah item: 3)
Producer menghasilkan: 92 (jumlah item: 4)
Consumer memproses : 92 (sisitem: 3)
Producer menghasilkan: 49 (jumlah item: 4)
Producer menghasilkan: 21 (jumlah item: 5)
Consumer memproses : 21 (sisitem: 4)
Producer menghasilkan: 62 (jumlah item: 5)
Producer menghasilkan: 27 (jumlah item: 6)
Consumer memproses : 27 (sisitem: 5)
Producer menghasilkan: 90 (jumlah item: 6)
Producer menghasilkan: 59 (jumlah item: 7)
Consumer memproses : 59 (sisitem: 6)
Producer menghasilkan: 63 (jumlah item: 7)
Producer menghasilkan: 26 (jumlah item: 8)
Consumer memproses : 26 (sisitem: 7)
Producer menghasilkan: 40 (jumlah item: 8)
Producer menghasilkan: 26 (jumlah item: 9)
Consumer memproses : 26 (sisitem: 8)
Producer menghasilkan: 72 (jumlah item: 9)
Producer menghasilkan: 36 (jumlah item: 10)
Consumer memproses : 36 (sisitem: 9)
Consumer memproses : 72 (sisitem: 8)
Consumer memproses : 40 (sisitem: 7)
Consumer memproses : 63 (sisitem: 6)
Consumer memproses : 90 (sisitem: 5)
Consumer memproses : 62 (sisitem: 4)
Consumer memproses : 49 (sisitem: 3)
Consumer memproses : 86 (sisitem: 2)
Consumer memproses : 93 (sisitem: 1)
Consumer memproses : 77 (sisitem: 0)

Produksi dan konsumsi selesai.
dea@LAPTOP-H54VQ765:~/Tugas1_KalkulatorParalel$ |
```

Program menunjukkan producer menghasilkan angka random dan menambahkannya ke buffer, sementara consumer mengambil angka untuk diproses. Jumlah item dalam buffer selalu sesuai kapasitas maksimal sehingga tidak pernah penuh atau kosong saat diakses. Sinkronisasi berjalan dengan baik, terlihat dari urutan produksi dan konsumsi yang tertib. Semua angka berhasil diproduksi dan dikonsumsi sesuai jumlah yang ditentukan.